

Programmation procédurale : les caractères



On parle de quoi ?

1. [Codification](#)
2. [Lecture](#)
3. [Le problème du tampon \(buffer\)](#)
4. [Impression](#)
5. [Fonctions](#)

Codification d'un caractère

```
void main(void) {  
    char c;  
    printf("Entrez un caractere :");  
    scanf_s("%c", &c, 1);  
  
    printf("\nCaractere %c", c);  
    printf("\nCode ASCII : %d", c);  
    printf("\nHexa : %X", c);  
}
```

Lecture d'un caractère

Lire un caractère peut se faire de deux manières différentes.

Avec `scanf_s`

```
void main(void) {  
    char c;  
    printf("Entrez un caractere :");  
    scanf_s("%c", &c, 1);  
}
```

Avec `getchar`

```
void main(void) {  
    char c;  
    printf("Entrez un caractere :");  
    c = getchar();  
}
```

Le problème du tampon (buffer)

Lors d'une lecture via `scanf_s`, l'entièreté des caractères entrés sur la console est enregistrée.

Ainsi, lorsque l'utilisateur appuie sur ENTER pour valider son entrée, le caractère de saut de ligne `\n` est également enregistré.

Après chaque `scanf_s` il reste donc un caractère à lire dans le tampon. Le `scanf` suivant (dans une boucle par exemple), utilisera ce caractère restant comme caractère à lire !

```
void main (void){  
    char reponse;  
    printf("Entrez un caractère :\n");  
    scanf_s("%c", &reponse, 1);  
    printf("Vous avez entré %c\n", reponse);  
  
    printf("Entrez un caractère :\n");  
    scanf_s("%c", &reponse,1);  
    printf("Vous avez entré %c\n", reponse);  
  
    printf("Une ligne vide est apparue comme par magie !");  
}
```

Le problème du tampon (buffer)

Pour résoudre ce problème, il suffit de "vider" le tampon en lisant tous les caractères restant.

```
void main (void){
    char reponse;
    printf("Entrez un caractère :\n");
    scanf_s("%c", &reponse,1);

    // ici, le buffer contient un \n

    printf("Vous avez entré %c\n", reponse);

    viderTampon(); // on vide le tampon : le \n disparaît du buffer

    printf("Entrez un caractère :\n");
    scanf_s("%c", &reponse, 1);
    printf("Vous avez entré %c\n", reponse);

    printf("La ligne vide a disparu !");
}
```

Le problème du tampon (buffer)

```
void viderTampon(void)
{
    int c;
    do
    {
        c = getchar();
    } while (c != '\n' && c != EOF);
}
```

Impression d'un caractère

L'impression d'un caractère peut se faire de deux manières différentes.

Avec `printf`

```
void main (void){
    char c;

    printf("Entrez un caractère :\n");
    scanf_s("%c", &c, 1);
    printf("Le caractère entré est %c\n", c);
}
```

Avec `putchar`

```
void main (void){
    char c;

    printf("Entrez un caractère :\n");
    scanf_s("%c", &c, 1); // ou c = getchar();
    printf("Le caractère entré est : ");
    putchar(c); // imprime c ET passe à la ligne
}
```


Fonctions

En C, il existe un grand nombre de fonctions prédéfinies pouvant être utilisées avec des caractères.

Fonctions de vérifications (`ctype.h`)

- `isalnum` : vérifie si un caractère est alphanumérique
- `isalpha` : vérifie si un caractère est une lettre de l'alphabet
- `isdigit` : vérifie si un caractère est un chiffre
- `islower` : vérifie si le caractère est une lettre de l'alphabet en minuscule
- `isupper` : vérifie si le caractère est une lettre de l'alphabet en majuscule
- `isspace` : vérifie si le caractère est un espace
- `isxdigit` : vérifie si le caractère fait partie du système hexadécimal

Fonctions de transformation (ctype.h)

- `tolower` : transforme le caractère en minuscule
- `toupper` : transforme le caractère en majuscule

Que fait le code suivant ?

```
char c;  
  
printf("Entrez un caractère : ");  
c = getchar();  
printf("Caractère transformé : ");  
putchar(toupper(++c));
```

Exercice : traduire le DA suivant.

```
┌── * Mon programme
│
│  Obtenir c
│  ┌── while(c ≠ '\n')
│  │  ┌── if (c = 'a')
│  │  │  c = 'z'
│  │  │  └── else if(c = 'A')
│  │  │  │  c = 'Z'
│  │  │  └── else
│  │  │  │  c devient la précédente
│  │  └──
│  └──
│  Sortir c
│  Obtenir c
└──
```